Relatório — Entrega 2: Janela Deslizante (GBN e SR)

CIC0236 – Teleinformática e Redes 2 – Turma 01 – 2026/2 – Universidade de Brasília

Grupo 10 (`-group grupo10`)

Anna Luiza Novaes Jansen Silva – 232024572

Vinicius Henrique Barbosa Silverio de Castro – 232006832

Código: `https://github.com/annalujansen/tr2-rdt-unb`

Todos os testes usam o arquivo `small` (1 MB, 2 048 segmentos de 512 B), timeout fixo de 500 ms e `-seed 42` (outras seeds só na verificação de robustez, ao fim da Seção 6).

1 Descrição da implementação

1.1 Go-Back-N (`gbn.py`)

O GBN mantém uma janela $[base, base + N)$ e **um único temporizador, o da base**. Não há buffer: só aceita o DATA da base.

Situação	Evento	Ação	Próximo estado
Enchendo a janela	`next_seq < base+N`	envia `REQ(next_seq)`; se `next_seq == base`, liga o timer	Enchendo / Esperando
Esperando DATA	DATA(`seq == base`)	envia ACK, guarda payload, grava linha do CSV, `base++`, reinicia timer	Enchendo a janela
Esperando DATA	DATA(`seq != base`)	descarta (fora de ordem ou duplicata)	Esperando DATA
Esperando DATA	timeout da base	marca a base com `timeout_occurred=1`, reinicia timer, reenvia `REQ(base ... next_seq-1)` (“volta N”)	Esperando DATA
Esperando DATA	`base == total`	calcula MD5, envia FIN	Encerrando

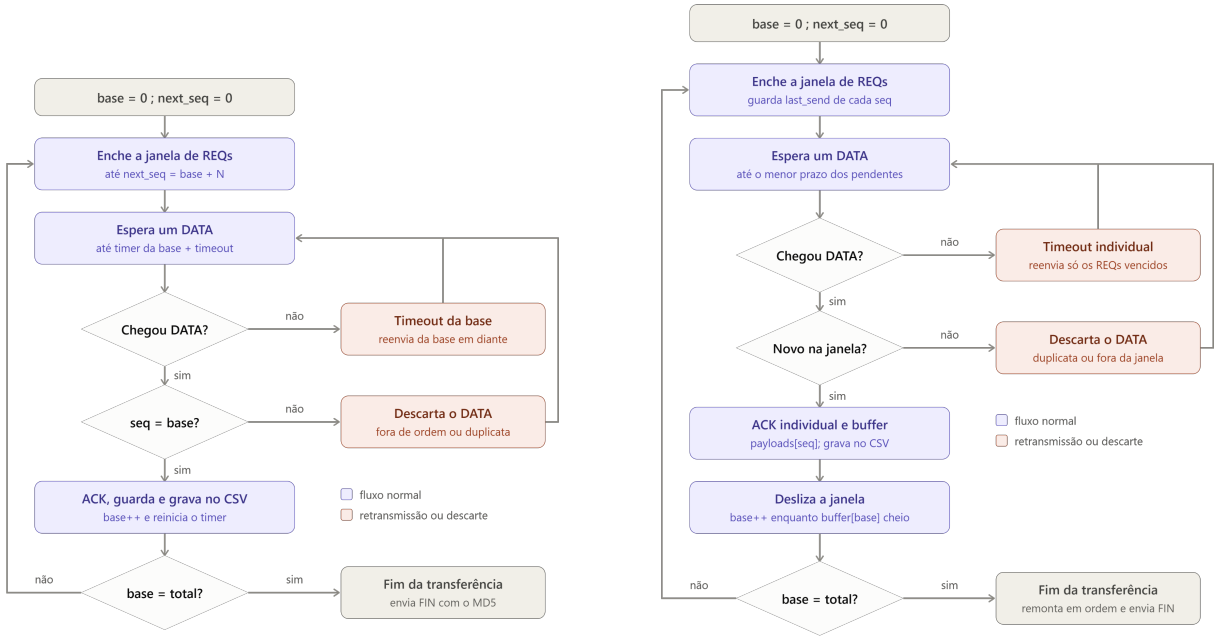
Critério de retransmissão: estouro do timer da base; reenvia a base e todos os seguintes, mesmo os que já tinham chegado (e foram descartados por estarem fora de ordem). No CSV, `timeout_occurred=1` marca só o segmento que estava na base (uma vez, ainda que a mesma base estoure mais de uma vez); os reenviados “de carona” aparecem apenas em `retransmissions`.

1.2 Selective Repeat (`sr.py`)

O SR mantém a janela $[base, base + N)$ e **um temporizador por segmento** (`last_send[seq] + timeout`). Como há um único socket, `recv_data` espera até o menor prazo entre os pendentes.

Situação	Evento	Ação	Próximo estado
Enchendo a janela	`next_seq < base+N`	envia `REQ(next_seq)` e registra `last_send`	Esperando DATA
Esperando DATA	DATA(`seq`) novo na janela	envia `ACK(seq)` individual, guarda em `payloads[seq]` (mesmo fora de ordem), grava linha do CSV	Avançando janela
Esperando DATA	DATA duplicado ou fora da janela	descarta	Esperando DATA
Esperando DATA	timeout de pendentes	reenvia `REQ` **só** dos segmentos cujo timer estourou	Esperando DATA
Avançando janela	`received[base]`	`base++` (enquanto houver consecutivos recebidos)	Enchendo a janela

Segmentos fora de ordem: ficam em `payloads[seq]` e `received[seq]`; o arquivo é remontado na ordem do `seq`, o que o MD5 confirma (Seção 3).



(a) Go-Back-N **(b)** Selective Repeat

Figura 1: Fluxogramas do cliente. Em vermelho, os caminhos de retransmissão e descarte: no GBN o timeout é só o da base e reenvia a janela inteira; no SR cada segmento tem seu timer e só os vencidos são reenviados.

2 Método de simulação de perda

Abordagem escolhida: (a), descartar o DATA recebido com probabilidade P (`-loss`). O DATA chega de verdade, mas, dentro de `comum.recv_data`, é descartado antes de qualquer processamento, como se a rede o tivesse perdido. O efeito no protocolo é o de uma perda real: o temporizador estoura e o REQ é reenviado.

Por que não a (b) (receber o DATA e não enviar o ACK): o servidor é *pull-based* e o cliente só avança ao receber o DATA. Deixar de enviar o ACK não provoca timeout nem retransmissão, então a (b) não simularia perda nenhuma neste protocolo.

Ponto único de aplicação: SAW, GBN e SR recebem DATA pelo mesmo `recv_data`, então a perda é aplicada de forma idêntica nos três. HELLO/OK e FIN/DONE não passam pela simulação.

Repetibilidade: o sorteio usa `random.Random(seed)`; todos os cenários oficiais usam `-seed 42`, registrada no `resumo_entrega2.csv`. Sem `-loss`, nenhum sorteio é feito.

Registro no CSV (colunas acrescentadas às da especificação):

Coluna	Significado
`simulated_loss`	1 se algum DATA deste segmento foi descartado pela simulação
`sim_drops`	quantos DATA deste segmento foram descartados
`recv_time_ms`	instante da aceitação desde o início da transferência

No SAW e no SR, uma retransmissão em segmento com `simulated_loss = 0` seria perda natural da rede UDP. Nos 21 cenários oficiais ela foi **zero**: em todos os cenários do SAW e do SR, o total de retransmissões é igual ao de DATA descartados pela simulação (91 com 5 %, 220 com 10 %), e os cenários sem perda simulada não tiveram nenhuma retransmissão. No GBN essa separação é diferente: os segmentos reenviados "de carona" no "volta N" também têm `simulated_loss = 0` (1 202 segmentos no GBN-16 com 10 %), sem que tenha havido perda natural.

Observação: o sorteio é feito **por DATA recebido**, não por segmento. No GBN cada segmento chega várias vezes (por causa do "volta N"), então ele consome mais sorteios e tem mais descartes que SAW e SR com a mesma seed (541 contra 220 com 10 %). A probabilidade por pacote é a mesma para todos.

3 Checksum MD5

Todos os 21 cenários (SAW, GBN e SR × janelas 4/8/16 × 0 %, 5 % e 10 %) terminaram com o mesmo MD5, igual ao informado pelo servidor no OK, e com `DONE ok=1`:

`f989450f6ca695013ceb6bd3a5f6417c`

O MD5 é o mesmo em todos porque o arquivo `small` do grupo 10 é sempre o mesmo. Isso confirma que a reordenação do SR e a remontagem do GBN estão corretas, mesmo com 10 % de perda.

4 Tabela comparativa

Cada célula traz **throughput médio (kbps) / total de retransmissões / tempo total (s)**, com throughput = file_size × 8/tempo_total. Cada cenário foi executado uma vez com `-seed 42`.

Protocolo	Janela	Sem perda	5 % de perda	10 % de perda
SAW	1	167,2 / 0 / 50,2	88,3 / 91 / 95,0	52,0 / 220 / 161,3
GBN	4	657,7 / 0 / 12,8	140,7 / 360 / 59,6	63,8 / 921 / 131,5
GBN	8	1235,5 / 0 / 6,8	152,0 / 744 / 55,2	66,9 / 1848 / 125,4
GBN	16	2025,5 / 0 / 4,1	159,5 / 1512 / 52,6	70,3 / 3562 / 119,3
SR	4	635,2 / 0 / 13,2	161,6 / 91 / 51,9	84,4 / 220 / 99,5
SR	8	1162,5 / 0 / 7,2	192,7 / 91 / 43,5	109,6 / 220 / 76,5
SR	16	1900,7 / 0 / 4,4	260,7 / 91 / 32,2	153,7 / 220 / 54,6

Tabela 1: Comparação SAW × GBN × SR: throughput médio (kbps) / retransmissões / tempo total (s).

Nota: o SAW sem perda atingiu 167 kbps aqui contra ∼64 kbps na Entrega 1 (`small`). A diferença se deve às condições da rede em cada dia (RTT médio de ∼24 ms agora contra ∼38 ms antes, com perda natural diferente).

5 Gráficos por segmento

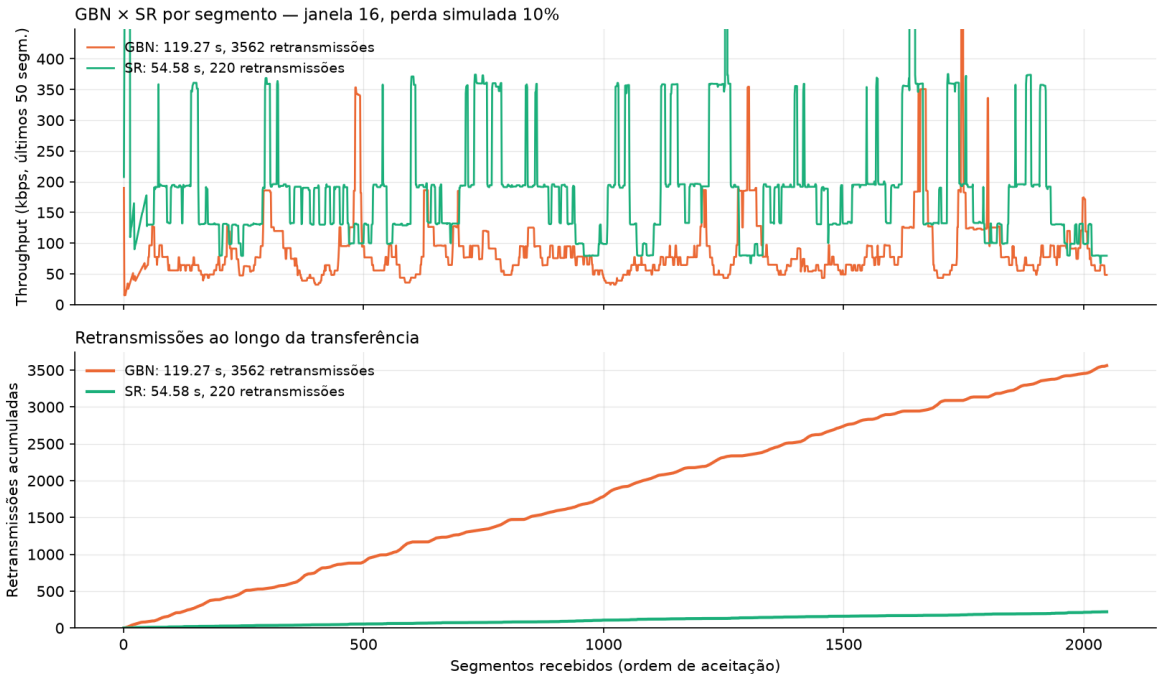


Figura 2: GBN × SR, janela 16, 10 % de perda: throughput (média dos últimos 50 segmentos) e retransmissões acumuladas ao longo da transferência. O GBN acumula retransmissões em saltos de até 16 a cada timeout; no SR a curva sobe de 1 em 1.

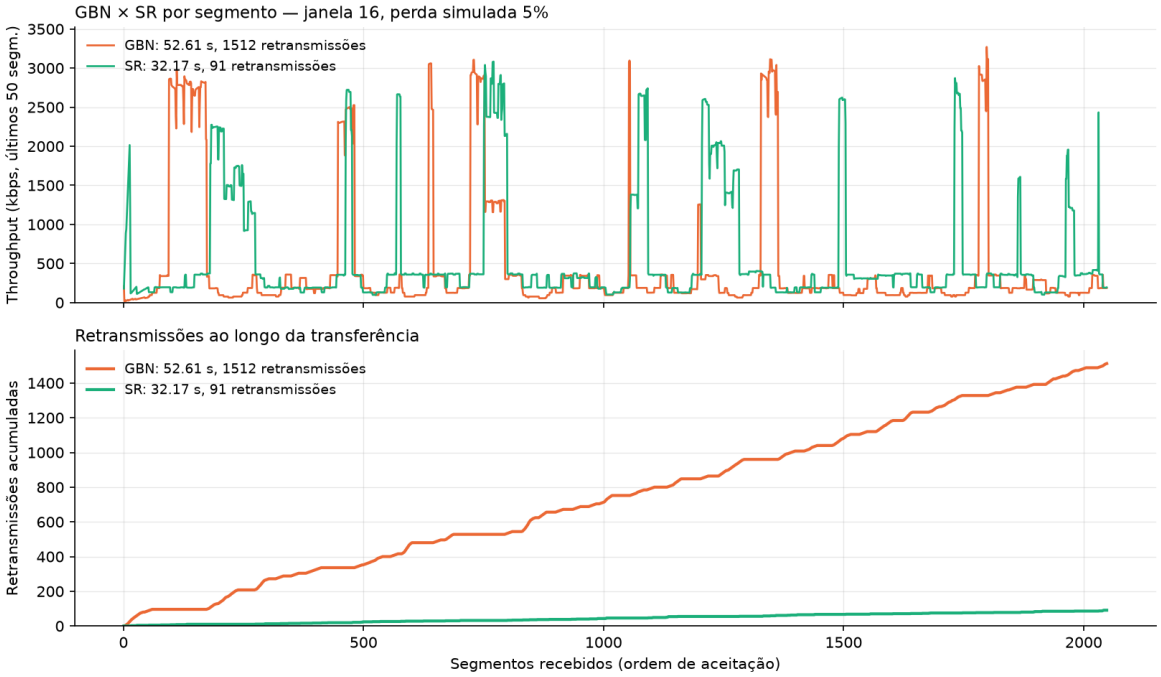


Figura 3: Mesma comparação com 5 % de perda.

6 Análise

a) Impacto da janela no throughput sem perda

Sem perda, a janela multiplica a vazão, pois throughput $\approx N \times L/\text{RTT}$: SAW 167 kbps $\rightarrow N = 4$ $\sim$650 $\rightarrow N = 8$ $\sim$1 200 $\rightarrow N = 16$ $\sim$2 000 kbps, com eficiência de $\sim$99 % em todos os casos (medido $\div$ teórico). O ganho com $N = 16$ é de $\sim$12$\times$ e não 16$\times$, porque o RTT médio sobe de $\sim$24 ms para $\sim$32–34 ms com 16 REQs em voo (enfileiramento). Sem perda, GBN e SR se comportam de forma praticamente igual ($N = 8$: 1 236 contra 1 163 kbps), pois a diferença entre eles só aparece quando há retransmissão.

Estabilização: nas janelas obrigatórias (4, 8 e 16) o throughput ainda cresce a cada dobra, com retorno decrescente (4$\rightarrow$8: 1,88$\times$; 8$\rightarrow$16: 1,64$\times$). Para achar o ponto de estabilização, rodamos uma varredura extra sem perda em 28/10 (GBN e SR, $N = 4, 8, 16, 24, 32, 48, 64, 96, 128$, duas rodadas cada, todas com MD5 correto; `resultados/janela_estabilizacao/`). A Figura 4 mostra o resultado: o throughput cresce quase linearmente até $N = 16$ ($\sim$2 000 kbps), desacelera em $N = 24$–32 ($\sim$2 300–2 500 kbps) e **estabiliza a partir de $N \approx 32$–48, em $\sim$2 700–2 900 kbps** (SR: 2 742 / 2 774 / 2 901 / 2 782 kbps em $N = 48/64/96/128$). Dobrar a janela de 48 para 96 rende só $\sim$6 %.

Por quê: enquanto $N \times L$ é menor que o produto banda $\times$ atraso, o cliente fica ocioso esperando DATA, e cada REQ a mais em voo vira vazão a mais, com o RTT praticamente constante ($\sim$30–33 ms até $N = 16$). Com um RTT base de $\sim$30 ms e um gargalo de $\sim$2,8 Mbps, esse produto é 2,8 Mbps $\times$ 30 ms $\approx$ 84 kbit $\approx$ 20 segmentos de 512 B: é exatamente onde a curva dobra. Acima disso o caminho (rede ou taxa de atendimento do servidor) já está sempre ocupado, e os REQs extras só esperam em fila: o RTT passa a crescer **proporcionalmente a N** (39 ms em $N = 24$, 50 ms em 32, 71 ms em 48, 93 ms em 64, $\sim$130 ms em 96), de modo que N/RTT fica constante em $\sim$700 segmentos/s ($\approx$ 1,4 ms por segmento). Ou seja, aumentar a janela além de $\sim$32 não aumenta a vazão, só a fila.

Janelas grandes demais chegam a piorar: em $N = 128$ a fila transborda e aparecem perdas naturais (sem `-loss`). O SR retransmite só os perdidos (64–73 retransmissões) e mantém $\sim$2 780 kbps, mas o GBN reenvia a janela inteira a cada timeout (604–724 retransmissões) e cai para $\sim$1 700 kbps. Perdas naturais esporádicas (1–2 timeouts) aparecem também em janelas menores, mas custam muito mais ao GBN (até 107 retransmissões em $N = 96$) que ao SR (no máximo 6). Para este servidor, uma janela em torno de **32–48** dá o throughput máximo sem encher a fila à toa.

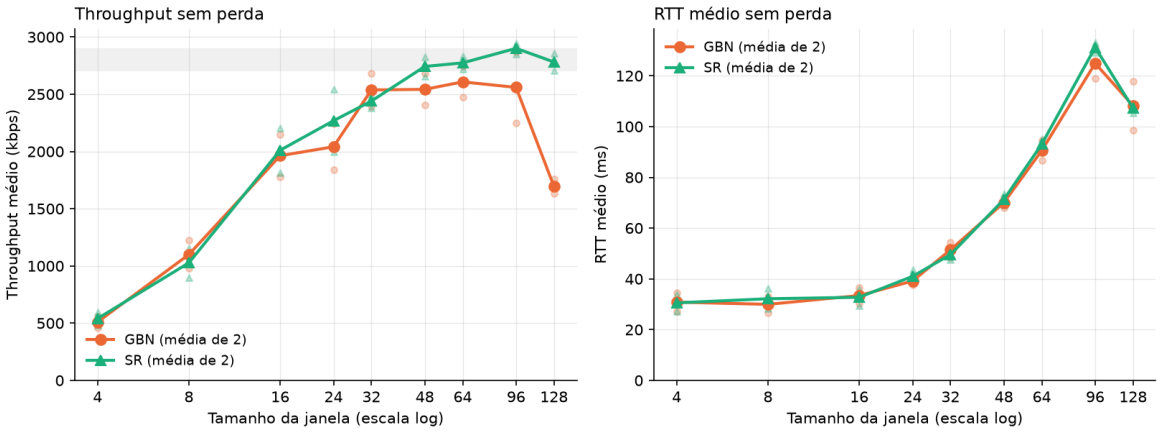


Figura 4: Varredura extra sem perda (28/10): throughput (esquerda) e RTT médio (direita) conforme a janela, em escala log. Pontos claros: cada rodada; linha: média das duas. A faixa cinza marca o patamar de $\sim$2 700–2 900 kbps. A partir de $N \approx 32$ o throughput para de crescer e o RTT passa a crescer na proporção de N (fila); em $N = 128$ surgem perdas naturais, que derrubam o GBN.

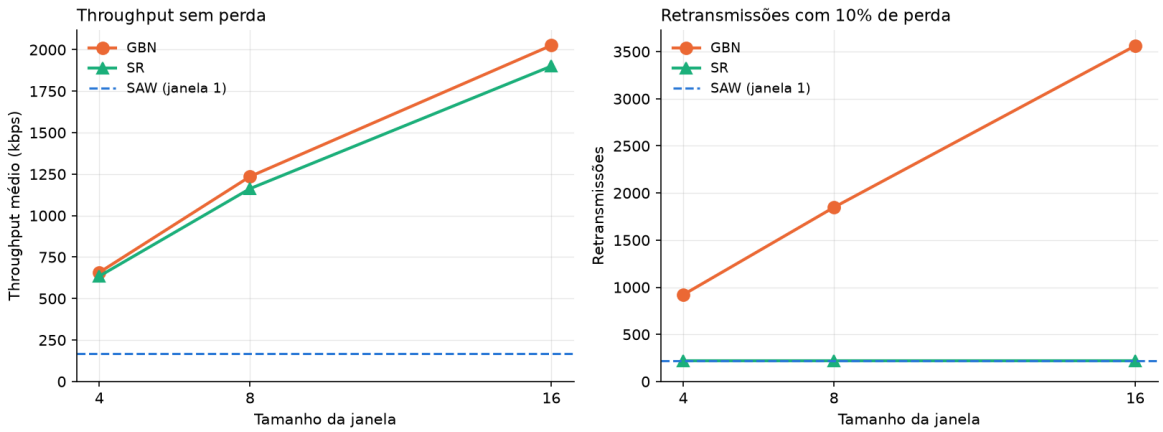


Figura 5: Esquerda: throughput sem perda conforme a janela (item a), com o SAW como referência. Direita: retransmissões com 10 % de perda conforme a janela (item b): no GBN crescem com N; no SR ficam iguais às do SAW.

b) Retransmissões por erro (10 % de perda, janela 16)

	Segm. com timeout	Timeouts (eventos)*	Retransmissões	Retx. ÷ timeout
GBN-16	207	$\geq$ 223	3 562	$\leq$ 16
SR-16	191	220	220	1,0

*No GBN, estouros do temporizador único da base; no SR, expirações dos temporizadores individuais de cada segmento.

A coluna `timeout_occurred` do CSV conta *segmentos* que sofreram timeout, não eventos: no GBN, se a mesma base estoura duas vezes, ela é marcada uma vez só. Por isso o número de eventos é obtido das retransmissões. No **GBN**, cada timeout reenvia a base e os segmentos seguintes já pedidos (no máximo 16), logo houve pelo menos $3\,562 \div 16 \approx 223$ timeouts. No CSV, 1 667 dos 2 048 segmentos foram retransmitidos ao menos uma vez (até 7 vezes), e 1 202 deles nunca tiveram um DATA próprio descartado. No **SR**, cada timeout reenvia exatamente um segmento: as 220 retransmissões coincidem com os 220 DATA descartados pela simulação (soma de `sim_drops`), em 191 segmentos (165 reenviados uma vez, 23 duas e 3 três vezes, quando o próprio reenvio também foi descartado). Contando por DATA descartado, o GBN fez $3\,562 \div 541 \approx 6{,}6$ retransmissões por descarte. Em números absolutos, o GBN-16 retransmitiu **16× mais** que o SR-16 nas mesmas condições de perda.

c) Por que o GBN sofre mais que o SR com perda alta

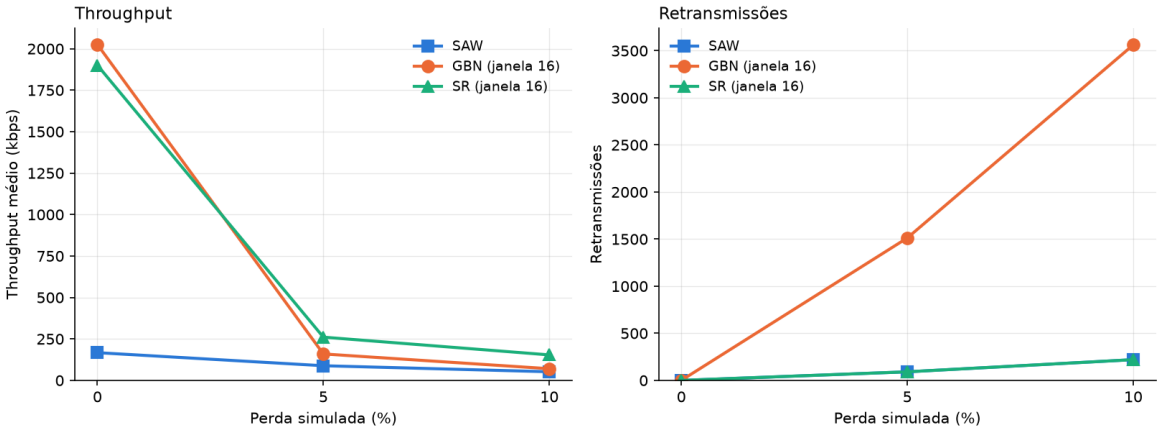


Figura 6: SAW, GBN-16 e SR-16 conforme a perda simulada: throughput médio (esquerda) e retransmissões (direita).

1. **Volume de retransmissões.** Com 10 % de perda, o GBN-16 retransmite 3 562 segmentos contra 220 do SR; com 5 %, 1 512 contra 91 (∼16,6×). O GBN reenvia a janela inteira a cada timeout, e cada reenvio extra é também um DATA exposto a novas perdas.
2. **Os timeouts do GBN se somam; os do SR se sobrepõem.** No GBN cada timeout trava todo o progresso: pelo menos $223 \times 0{,}5\,\text{s} \approx 112\,\text{s}$ dos 119,3 s do GBN-16 foram gastos esperando timeouts. No SR, enquanto um segmento espera o timeout, os outros da janela continuam chegando e vão para o buffer: os 220 timeouts do SR-16 somariam 110 s se fossem sequenciais, mas a transferência terminou em **54,6 s**. Contando no CSV os intervalos de mais de 300 ms sem nenhuma chegada, o SR-16 teve só 98 paradas para 220 timeouts (vários timers da mesma janela vencem juntos e uma única espera recupera várias perdas), enquanto o GBN-16 teve 207.
3. **A janela só ajuda o SR com perda.** Com 10 % de perda, o throughput do GBN quase não cresce com N (63,8 → 66,9 → 70,3 kbps), enquanto o do SR cresce bastante (84,4 → 109,6 → 153,7 kbps): quanto maior N, mais trabalho útil acontece durante cada espera por timeout. O SR-16 com 10 % de perda (153,7 kbps) chega perto do SAW *sem* perda (167,2 kbps), enquanto o GBN-16 com 10 % (70,3 kbps) fica abaixo da metade.

Robustez à seed: para verificar que o resultado não depende da sequência sorteada, repetimos GBN-16 e SR-16 com 10 % de perda e as seeds 7 e 123 (em `resultados/robustez_seed/`, todas com MD5 correto):

Seed	Retx. GBN-16	Retx. SR-16	GBN ÷ SR	Tempo GBN-16 (s)	Tempo SR-16 (s)
42	3 562	220	16,2×	119,3	54,6
7	3 825	231	16,6×	127,7	51,7
123	3 634	248	14,7×	121,8	54,9

Nas três seeds o GBN retransmitiu ∼15–17× mais e levou mais que o dobro do tempo do SR. Na seed 123 o SR teve 248 retransmissões para 247 DATA descartados pela simulação: a única retransmissão sem `simulated_loss` é uma perda natural da rede, separada justamente por esse campo.